

Threads & Concurrency

Practice Exercises

- 4.1 Provide three programming examples in which multithreading provides better performance than a single-threaded solution.

Answer:

- A web server that services each request in a separate thread
- A parallelized application such as matrix multiplication where various parts of the matrix can be worked on in parallel
- An interactive GUI program such as a debugger where one thread is used to monitor user input, another thread represents the running application, and a third thread monitors performance

- 4.2 Using Amdahl's Law, calculate the speedup gain of an application that has a 60 percent parallel component for (a) two processing cores and (b) four processing cores.

Answer:

$$\text{Parallel speedup} = 1 / \{(1-P) + P/n\} \quad \text{Here, } P = 0.6$$

- With two processing cores, we get a speedup of 1.42 times.
- With four processing cores, we get a speedup of 1.82 times.

- 4.3 Does the multithreaded web server described in Section 4.1 exhibit task or data parallelism?

Answer:

Data parallelism. Each thread is performing the same task but on different data.

- 4.4 What are two differences between user-level threads and kernel-level threads? Under what circumstances is one type better than the other?

Answer:

- User-level threads are unknown by the kernel, whereas the kernel is aware of kernel threads.
- On systems using either many-to-one or many-to-many model mapping, user threads are scheduled by the thread library, and the kernel schedules kernel threads.

Creation and management: User-level threads are created and managed by the application itself, while kernel-level threads are created and managed by the operating system. This means that user-level threads are typically faster to create and switch between, but kernel-level threads have more control over the hardware.

Isolation: User-level threads share the same address space, while kernel-level threads have their own separate address spaces. This means that user-level threads are not as isolated from each other as kernel-level threads, which can lead to problems if one thread crashes.

Under what circumstances is one type better than the other?

User-level threads are better for applications that need to be fast and responsive. This is because user-level threads are typically faster to create and switch between, which can improve the performance of the application. Kernel-level threads are better for applications that need to be highly concurrent or that need to access hardware resources directly. This is because kernel-level threads have more control over the hardware, which can improve the performance of the application. Ultimately, the best type of thread to use depends on the specific application and its requirements.

- c. Kernel threads need not be associated with a process, whereas every user thread belongs to a process. Kernel threads are generally more expensive to maintain than user threads, as they must be represented with a kernel data structure.

Describe the actions taken by a kernel to context-switch between kernel-level threads.

Answer:

Context switching between kernel threads typically requires saving the value of the CPU registers from the thread being switched out and restoring the CPU registers of the new thread being scheduled.

What resources are used when a thread is created? How do they differ from those used when a process is created?

Answer:

Because a thread is smaller than a process, thread creation typically uses fewer resources than process creation. Creating a process requires allocating a process control block (PCB), a rather large data structure. The PCB includes a memory map, a list of open files, and environment variables. Allocating and managing the memory map is typically the most time-consuming activity. Creating either a user thread or a kernel thread involves allocating a small data structure to hold a register set, stack, and priority.

Assume that an operating system maps user-level threads to the kernel using the many-to-many model and that the mapping is done through LWPs. Furthermore, the system allows developers to create real-time threads for use in real-time systems. Is it necessary to bind a real-time thread to an LWP? Explain.

light-weight process (LWP)

Answer:

Yes. Timing is crucial to real-time applications. If a thread is marked as real-time but is not bound to an LWP, the thread may have to wait to be attached to an LWP before running. Consider a situation in which a real-time thread is running (is attached to an LWP) and then proceeds to block (must perform I/O, has been preempted by a higher-priority real-time thread, is waiting for a mutual exclusion lock, etc.). While the real-time thread is blocked, the LWP it was attached to is assigned to another thread. When the real-time thread has been scheduled to run again, it must first wait to be attached to an LWP. By binding an LWP to a real-time thread, you are ensuring that the thread will be able to run with a minimal delay once it is scheduled.

10E Exercises

- 4.8 Provide two programming examples in which multithreading does not provide better performance than a single-threaded solution.

Answer:

- a. Any kind of sequential program is not a good candidate to be threaded. An example of this is a program that calculates an individual tax return.
- b. Another example is a "shell" program such as the C-shell or Korn shell. Such a program must closely monitor its own working space such as open files, environment variables, and current working directory.

A multithreaded solution using multiple kernel threads can provide better performance than a single-threaded solution on a single-processor system under the following circumstances:

The application is CPU-intensive. If the application is spending most of its time executing CPU-intensive code, then a multithreaded solution can improve performance by allowing the CPU to switch between threads that are waiting for resources.

The application is I/O-bound. If the application is spending most of its time waiting for I/O, then a multithreaded solution can improve performance by allowing the CPU to switch to other threads that are not waiting for I/O.

The application is using a lot of system resources. If the application is using a lot of system resources, such as memory or the CPU, then a multithreaded solution can improve performance by allowing the operating system to better manage these resources.

- 4.9 Under what circumstances does a multithreaded solution using multiple kernel threads provide better performance than a single-threaded solution on a single-processor system?

Answer:

When a kernel thread suffers a page fault, another kernel thread can be switched in to use the interleaving time in a useful manner. A single-threaded process, on the other hand, will not be capable of performing useful work when a page fault takes place. Therefore, in scenarios where a program might suffer from frequent page faults or has to wait for other system events, a multithreaded solution would perform better even on a single-processor system.

- 4.10 Which of the following components of the program state are shared across threads in a multithreaded process?
- a. Register values
 - b. Heap memory
 - c. Global variables
 - d. Stack memory

Answer:

The threads of a multithreaded process share heap memory and global variables. Each thread has its separate set of register values and a separate stack.

- 4.11 Can a multithreaded solution using multiple user-level threads achieve better performance on a multiprocessor system than on a single-processor system? Explain.

Answer:

A multithreaded system comprising multiple user-level threads cannot make use of the different processors in a multiprocessor system simultaneously. The operating system sees only a single process and will not schedule the different threads of the process on separate processors. Consequently, there is no performance benefit associated with executing multiple user-level threads on a multiprocessor system.

- 4.12 In Chapter 3, we discussed Google's Chrome browser and its practice of opening each new tab in a separate process. Would the same benefits have been achieved if, instead, Chrome had been designed to open each new tab in a separate thread? Explain.

Answer:

No. The primary reason for opening each website in a separate process is that if a web application in one website crashes, only that renderer process is affected, and the browser process, as well as other renderer processes, are unaffected. Because multiple threads all belong to the same process, any thread that crashes would affect the entire process.

4.13 Is it possible to have concurrency but not parallelism? Explain.

Answer:

Yes. Concurrency means that more than one process or thread is progressing at the same time. However, it does not imply that the processes are running simultaneously. The scheduling of tasks allows for concurrency, but parallelism is supported only on systems with more than one processing core.

4.14 Using Amdahl's Law, calculate the speedup gain for the following applications:

- 40 percent parallel with (a) eight processing cores and (b) sixteen processing cores
- 67 percent parallel with (a) two processing cores and (b) four processing cores
- 90 percent parallel with (a) four processing cores and (b) eight processing cores

Answer:

$$\text{Parallel speedup} = 1 / \{(1-P) + P/n\}$$

- $P = 0.4$ parallel with (a) eight processing cores = 1.54 speedup and (b) sixteen processing cores = 1.6 speedup
- $P = 0.67$ parallel with (a) two processing cores = 1.5 speedup and (b) four processing cores = 2.01 speedup
- $P = 0.9$ parallel with (a) four processing cores = 3.08 speedup and (b) eight processing cores = 4.71 speedup

4.15 Determine if the following problems exhibit task or data parallelism:

- Using a separate thread to generate a thumbnail for each photo in a collection
- Transposing a matrix in parallel
- A networked application where one thread reads from the network and another writes to the network
- The fork-join array summation application described in Section 4.5.2
- The Grand Central Dispatch system

Answer:

- Task parallelism. Each thread is running the same code, but on entirely different photo data.
- Data parallelism. Each thread is performing the same task on different subsets of data.
- Task parallelism. Each thread is performing a different task on the same data.
- Data parallelism. Each thread is performing the same task on different subsets of data.
- Task parallelism. Each thread is performing a different task on the same set of data.

4.16 A system with two dual-core processors has four processors available for

scheduling. A CPU-intensive application is running on this system. All input is performed at program start-up when a single file must be opened. Similarly, all output is performed just before the program terminates, when the program results must be written to a single file. Between start-up and termination, the program is entirely CPU-bound. Your task is to improve the performance of this application by multithreading it. The application runs on a system that uses the one-to-one threading model (each user thread maps to a kernel thread).

- How many threads will you create to perform the input and output? Explain.
- How many threads will you create for the CPU-intensive portion of the application? Explain.

Answer:

- It only makes sense to create as many threads as there are blocking system calls, as the threads will be spent blocking. Creating additional threads provides no benefit. Thus, it makes sense to create a single thread for input and a single thread for output.
- Four. There should be as many threads as there are processing cores. Fewer would be a waste of processing resources, and any number > 4 would be unable to run.

4.17 Consider the following code segment:

```
pid_t pid;
pid = fork();
if (pid == 0) { /* child process */
    fork();
    thread_create( . . . );
}
fork();
```

- a. How many unique processes are created?
- b. How many unique threads are created?

Answer:

There are six processes and two threads.

4.18 As described in Section 4.7.2, Linux does not distinguish between processes and threads. Instead, Linux treats both in the same way, allowing a task to be more akin to a process or a thread depending on the set of flags passed to the `clone()` system call. However, other operating systems, such as Windows, treat processes and threads differently. Typically, such systems use a notation in which the data structure for a process contains pointers to the separate threads belonging to the process. Contrast these two approaches for modeling processes and threads within the kernel.

Answer:

On one hand, in systems where processes and threads are considered similar entities, some of the operating system code could be simplified. A scheduler, for instance, can consider the different processes and threads on an equal footing without requiring special code to examine the threads associated with a process during every scheduling step. On the other hand, this uniformity could make it harder to impose process-wide resource constraints in a direct manner. Instead, some extra complexity is required to identify which threads correspond to which process and perform the relevant accounting tasks.

```
#include <pthread.h>
#include <stdio.h>

int value = 0;
void *runner(void *param); /* the thread */

int main(int argc, char *argv[])
{
    pid_t pid;
    pthread_t tid;
    pthread_attr_t attr;
    pid = fork();

    if (pid == 0) { /* child process */
        pthread_attr_init(&attr);
        pthread_create(&tid, &attr, runner, NULL);
        pthread_join(tid, NULL);
        printf("CHILD: value = %d", value); /* LINE C */
    }
    else if (pid > 0) { /* parent process */
        wait(NULL);
        printf("PARENT: value = %d", value); /* LINE P */
    }
}

void *runner(void *param)
{
    value = 5;
    pthread_exit(0);
}
```

Figure 4.23 C program for Exercise 4.19.

- 4.19** The program shown in Figure 4.23 uses the Pthreads API. What would be the output from the program at LINE C and LINE P?

Answer:

Output at LINE C is 5. Output at LINE P is 0.

- 4.20 Consider a multicore system and a multithreaded program written using the many-to-many threading model. Let the number of user-level threads in the program be greater than the number of processing cores in the system. Discuss the performance implications of the following scenarios.
- The number of kernel threads allocated to the program is less than the number of processing cores.
 - The number of kernel threads allocated to the program is equal to the number of processing cores.
 - The number of kernel threads allocated to the program is greater than the number of processing cores but less than the number of user-level threads.

Answer:

When the number of kernel threads is less than the number of processors, then **some of the processors would remain idle** since the scheduler maps only kernel threads to processors and not user-level threads to processors. When the number of kernel threads is exactly equal to the number of processors, then it is possible that **all of the processors might be utilized simultaneously**. However, when a kernel-thread block inside the kernel (due to a page fault or while invoking system calls), the corresponding processor would remain idle. When there are more kernel threads than processors, a blocked kernel thread could be swapped out in favor of another kernel thread that is ready to execute, thereby increasing the utilization of the multiprocessor system.

- 4.21 Pthreads provides an API for managing thread cancellation. The `pthread_setcancelstate()` function is used to set the cancellation state. Its prototype appears as follows:

```
pthread_setcancelstate(int state, int *oldstate)
```

The two possible values for the state are `PTHREAD_CANCEL_ENABLE` and `PTHREAD_CANCEL_DISABLE`. Using the code segment shown in Figure 4.24, provide examples of two operations that would be suitable to perform between the calls to disable and enable thread cancellation.

```
int oldstate;
pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, &oldstate);
/* What operations would be performed here? */
pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, &oldstate);
```

Figure 4.24 C program for Exercise 4.21.

Answer:

Three examples:

- An update to a file

- b. A situation in which two write operations must both be complete if either completes
- c. Essentially any operation that we want to run to completion

9E Exercises

4.13 Determine if the following problems exhibit task or data parallelism:

- The multithreaded statistical program described in Exercise 4.21
- The multithreaded Sudoku validator described in Project 1
- The multithreaded sorting program described in Project 2
- The multithreaded web server described in Section 4.1

Answer:

- Task parallelism. Each thread is performing a different task on the same set of data.
- Task parallelism. Each thread is performing a different task on the same data.
- Data parallelism. Each thread is performing the same task on different subsets of data.
- Task parallelism. Likely running the same code, but on entirely different data.

Solution by D. M. Nadim Hayder